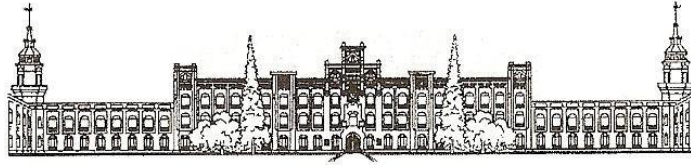


Міністерство освіти і науки України
Національний технічний університет
України
«Київський політехнічний інститут імені Ігоря Сікорського»



Факультет прикладної математики

Кафедра системного програмування і спеціалізованих комп'ютерних систем

Лабораторна робота № 4

з дисципліни

«Різницеві моделі цифрових пристроїв. Моделювання динамічного нейрону. Графічне дослідження моделі»

Виконала студентка групи:

КВ-11 Михайліченко Софія

Перевірив: Малезик П.М.

Оцінка:

Мета: Провести моделювання СП, які описують різницеви́ми рівня́ми. Ознайомитися з деякими можливостями графічного дослідження процесів перетворення інформації дисипативними динамічними системами. Дослідити моделі нейронів з хаотичною поведінкою.

Завдання:

1. Розробити алгоритм та написати програму в системі Matlab для розв'язання рівняння

$$x_{n+1} = 4x_n(1+x_n)$$

2. Побудувати діаграму Ламерея та знайти нерухомі стійкі точки

3. Вивчити основні типи нейронів, які представлені в підсистемі Neural Networks пакету Matlab

Завдання 1

Розділимо алгоритм для написання програми в системі Matlab для розв'язання рівняння на пункти та детально проаналізуємо кожен з них:

Пункт 1: Розуміння рівняння

Перш ніж реалізувати алгоритм, важливо зрозуміти природу та особливості рівняння $x_{n+1}=4x_n(1+x_n)$. Це рекурсивне рівняння відображає процес, де кожен наступний член послідовності x_{n+1} обчислюється на основі попереднього члена x_n . Воно відображає динаміку системи, де значення збільшуються швидше за лінійно. Таке рівняння може виникнути, наприклад, у задачах моделювання росту популяції або в реакційних кінетичних рівняннях. Обчислення наступних значень x_n за допомогою цього рівняння дозволяє наближено знаходити розв'язок або аналізувати поведінку системи протягом часу. Розуміння фізичного або математичного контексту рівняння допомагає ефективніше використовувати розроблений алгоритм та аналізувати його результати.

Пункт 2: *Визначення початкових умов*

На початку програми визначається значення x_0 - початкова точка, з якої розпочинається ітерація, а також визначається кількість ітерацій, необхідних для обчислення. Наприклад: $x_0 = 0.001$; iterations = 8;.Вибір початкового значення і кількості ітерацій може залежати від конкретної задачі та потреб користувача, тому їх слід обирати з урахуванням контексту задачі.

Пункт 3: *Ініціалізація масивів*

Після визначення початкового значення та кількості ітерацій, необхідно створити два масиви: один для зберігання номерів ітерацій X та інший для зберігання значень послідовності L . Це допоможе зручно зберігати та доступатися до результатів обчислень на різних етапах програми. Ініціалізація масивів заздалегідь дозволяє уникнути можливих проблем з індексацією під час обчислень.

Пункт 5: *Обчислення послідовності*

Для обчислення значень послідовності запусимо цикл for, який буде ітеруватися від 1 до iterations – 1. Це дозволить обчислити значення x_{n+1} для кожної ітерації. У кожній ітерації використаєм формулу $x_{n+1} = 4x_n(1+x_n)$ для обчислення нового значення x_{n+1} на основі попереднього значення x_n . Запишемо обчислене значення x_{n+1} в масив L , щоб зберегти його для подальшого використання. Також, запишемо номер поточної ітерації в масив X , щоб відслідковувати порядковий номер кожного обчисленого значення.

Пункт 6: *Виведення результатів*

Після завершення усіх ітерацій виведемо масиви X і L , щоб перевірити обчислені значення та переконатися, що програма працює правильно. Результати можна вивести на екран або зберегти у файл для подальшого аналізу або використання.

Пункт 7: *Побудова графіка*

Після виведення результатів можна побудувати графік залежності значень послідовності L від номеру ітерації X за допомогою функції plot(). Додамо підписи до осей графіка, а також заголовок, щоб зрозуміти, що саме

відображено на графіку. Включимо сітку на графіку, щоб краще виділити точки та лінії графіка і полегшити їх інтерпретацію.

```
>> % Початкове значення
x0 = 0.001;
% Кількість ітерацій
iterations = 8;
% Ініціалізація масивів
X = zeros(1, iterations);
L = zeros(1, iterations);
% Початкове значення
L(1) = x0;
% Обчислення значень
for i = 1:iterations-1
    X(i) = i;
    L(i+1) = 4 * L(i) * (1 + L(i));
end
X(iterations) = iterations;
% Виведення результатів
disp('X:');
disp(X);
disp('L:');
disp(L);
% Побудова графіка
figure;
plot(X, L, '-o', 'LineWidth', 2, 'MarkerSize', 4);
title('Графік послідовності L');
xlabel('Номер ітерації');
ylabel('Значення L');
grid on;
```

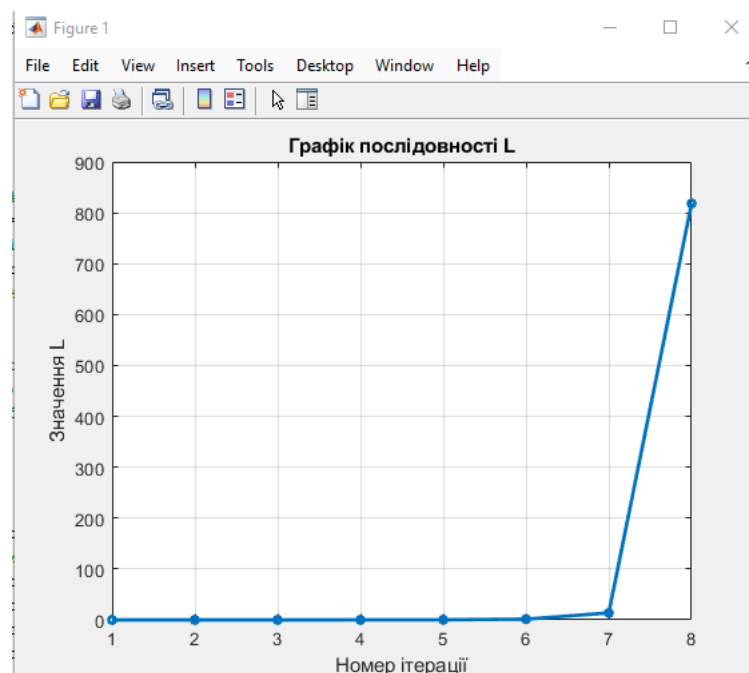
X:

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

L:

0.0010	0.0040	0.0161	0.0654	0.2785	1.4243	13.8113	818.2575
--------	--------	--------	--------	--------	--------	---------	----------

Приклад роботи програми



Завдання 2

Діаграма Ламеля (також відома як діаграма кобвеб або графік кобвеб) - це графічний метод візуалізації динаміки ітераційного процесу. Вона використовується для графічного відображення послідовності значень, які генеруються за допомогою ітераційної функції.

Основна ідея діаграми Ламеля полягає у візуальному представленні значень $X(n)$ та $X(n+1)$, де $X(n)$ є поточним значенням, а $X(n+1)$ - наступним значенням, яке обчислюється за допомогою ітераційної функції. Графік показує точки, що відповідають цим значенням, та лінії, які з'єднують кожен точку $X(n)$ з відповідною точкою $X(n+1)$.

Діаграма Ламеля допомагає візуалізувати збіжність або розбіжність ітераційного процесу, спостерігати за його поведінкою, виявляти, а також визначати стійкість точок фіксації або періодичних точок в системі.

Такий графічний підхід дозволяє легше розуміти динаміку ітераційного процесу та його вплив на систему, що досліджується, і забезпечує більш глибоке розуміння властивостей цієї системи.

```
function cobweb(x0, tol, a, b, n)
    format compact;
    fs = 25;
    lw = 2;

    % Генеруємо значення X
    xx = linspace(a, b, n);

    % Визначаємо функцію
    cub = @(x) 4*x.*(1+x);

    % Обчислюємо відповідні значення Y
    w = cub(xx);
    yy = xx;

    % Побудова вихідної функції
    figure(1);
    hold on;
    set(gca, 'DefaultLineLineWidth', lw);
    set(gca, 'FontSize', fs);
    xlabel('X(n)'); % Вісь X
    ylabel('X(n+1)'); % Вісь Y
    plot(xx, w, 'b', xx, yy, 'g', xx, xx*0, 'k', xx*0, xx, 'k');

    % Ініціалізуємо ітерацію
    i = 1;
    x(i) = x0;
    x(i+1) = cub(x(i));
```

```

% Побудова початкової точки
plot([x(i), x(i)], [0, x(i+1)], 'r');
fprintf('x(%d)=%1.20f\n', i, x(i));

% Ітераційний цикл
while (((abs(x(i+1)-x(i)) > tol && abs(x(i+1)) < 3) || i < 5) && min(abs(x(end)-
x(1:end-1))) > tol)
    i = i + 1;
    x(i+1) = cub(x(i));

    % Побудова ліній та точки для поточної ітерації
    plot([x(i-1), x(i)], [x(i), x(i)], 'r');
    plot([x(i), x(i)], [x(i), x(i+1)], 'r');
    fprintf('x(%d)=%1.20f\n', i, x(i));

    % Адаптація вісі
    axis auto;

    % Вихід з циклу (за потреби)
    break;
end

title('діаграма Ламерея', 'FontWeight', 'normal');
end

```

Проаналізуємо написану програму:

Функція **cobweb** призначена для побудови діаграми Ламеля, що є графічним методом візуалізації ітераційного процесу. Основна мета - візуально відобразити динаміку послідовності значень, які генеруються за допомогою заданої ітераційної функції.

Вхідні параметри функції включають:

- **x0**: початкове значення X для ітераційного процесу.
- **tol**: допустима різниця між сусідніми значеннями при ітерації, яка визначає точність обчислень.
- **a, b**: ліва та права границі діапазону значень X .
- **n**: кількість проміжних значень X між **a** та **b**.

Початкове значення X та інші параметри використовуються для генерації послідовності значень $X(n)$. Далі ці значення обчислюються за допомогою заданої ітераційної функції $f(x)=4x(1+x)$, яка передається у вигляді анонімної функції.

На початку функція генерує значення X за допомогою **linspace** в заданому діапазоні від **a** до **b**. Далі обчислюються відповідні значення ітераційної функції для кожного значення X .

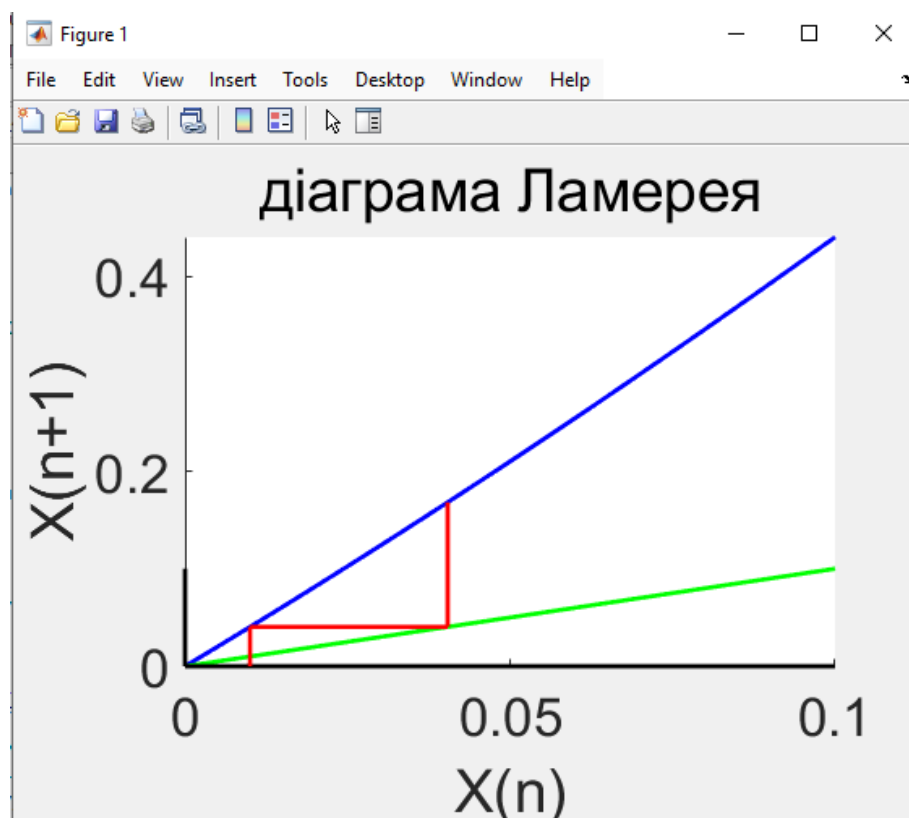
Графічна частина функції включає побудову графіку, де на осях відображаються значення $X(n)$ та $X(n+1)$. Крім того, на графіку відображається графік функції $f(x)$, пряма $y=x$, а також вісі X та Y .

Після побудови графіку ініціалізується ітераційний процес, який виконується у циклі. Кожна ітерація обчислює наступне значення $X(n+1)$ та відображає його на графіку разом з відповідною лінією, що з'єднує $X(n)$ та $X(n+1)$. Адаптація вісі виконується автоматично для покращення візуального представлення.

Щоб перевірити написання програми у командному рядку MatLab вводимо

```
>> cobweb(0.001,1e-20,0,1,100)
```

Приклад роботи програми



Завдання 3

В пакеті Neural Networks пакету Matlab доступні різні типи нейронів, які можуть бути використані для побудови широкого спектру нейронних мереж. Різноманіття моделей нейронних мереж і безліч параметрів, які необхідно встановити (розміри мережі, параметри алгоритму навчання і т.д.), може поставити іншого користувача в безвихідь. Саме для цього в пакеті існує Майстер рішень, який може автоматично провести пошук відповідної архітектури мережі будь-якої складності. У системі STATISTICA Neural Networks реалізовані всі основні типи нейронних мереж, що використовуються при рішенні практичних задач (Загалом існує три різних типи нейронів: збуджуючі, гальмуючі та комбіновані (себто, ті які можуть бути і збуджуючими, і гальмуючими)), зокрема: багатошарові персептрони, мережі на радіальних базисних функціях, карти Кохонена, імовірнісні нейронні мережі; мережі для кластеризації, лінійні мережі і т. д.

Ось деякі з них:

Персептрони (Perceptrons):

Характеристика: Простий тип нейронних мереж, який складається з одного шару нейронів, без прихованих шарів. Кожен нейрон в цьому шарі пов'язаний з кожним входом і має вагу, яку можна навчити або налаштувати.

Використання: Використовується для бінарної класифікації або простих лінійних регресійних завдань, коли дані можуть бути лінійно розділені.

Штучні нейронні мережі (Artificial Neural Networks, ANN):

Характеристика: Мережі, які складаються з багатьох нейронів, організованих у велику кількість шарів, включаючи вхідний, прихований і вихідний. Мережі можуть мати різні архітектури, включаючи повнозв'язні мережі та мережі з прихованими шарами.

Використання: Застосовується для різноманітних завдань машинного навчання, включаючи класифікацію, регресію, апроксимацію функцій та інше.

Зворотні нейронні мережі (Recurrent Neural Networks, RNN):

Характеристика: Мережі, які мають зв'язки, що формують цикли, що дозволяють їм моделювати послідовності даних. Кожен нейрон у RNN приймає вхід і власний вихід, а також вхід від попереднього часового кроку.

Використання: Широко використовуються для аналізу тексту, мовлення, часових рядів та інших послідовних даних, де важливий контекст та послідовність.

Конволюційні нейронні мережі (Convolutional Neural Networks, CNN):

Характеристика: Мережі, спеціалізовані на обробці зображень, використовуючи конволюції для виявлення ознак та підсумовуванням для зменшення розмірності даних.

Використання: Зазвичай використовується в завданнях комп'ютерного зору, відеоаналітики, обробки медичних зображень та інших областей, де важливі просторові структури даних.

Мережі з довготривалим короткостроковою пам'яттю (Long Short-Term Memory, LSTM):

Характеристика: Спеціальний тип RNN, який здатний ефективно вирішувати завдання, пов'язані з обробкою послідовностей даних, зокрема мовлення та машинний переклад, зберігаючи інформацію впродовж тривалого часу.

Використання: Широко використовується в машинному перекладі, генерації тексту, аналізі відео та інших послідовних завданнях.

Мережі типу GAN (Generative Adversarial Networks):

Характеристика: Мережі, які складаються з двох моделей: генератора, який створює зразки, і дискримінатора, який намагається розрізнити між реальними та створеними генератором зразками. Це принципово новий підхід до генерації даних, що забезпечує високу якість згенерованих зразків.

Використання: Використовуються для генерації нових зображень, аудіо, тексту та інших типів даних, а також для вирішення проблем генерації даних у відсутність великих наборів даних.

Висновок

Ця лабораторна робота включає кілька етапів, які спрямовані на глибоке розуміння та вивчення різних аспектів аналізу та моделювання динамічних систем за допомогою Matlab.

На першому етапі студенти розробляють алгоритм та пишуть програму для розв'язання заданого диференціального рівняння. Це вимагає ретельного розуміння рівняння та обрання відповідних методів чисельного розв'язання.

Далі, вони вивчають побудову діаграми Ламерея, що є потужним інструментом для визначення стійкості розв'язків систем диференціальних рівнянь. Цей етап дозволяє студентам візуалізувати та аналізувати стійкість розв'язків системи, а також знаходити нерухомі стійкі точки.

Останній етап лабораторної роботи передбачає вивчення основних типів нейронів у пакеті Neural Networks Matlab. Це дозволяє студентам розширити свої знання про штучний інтелект та нейромережі, а також здобути практичні навички у використанні цих інструментів для аналізу та моделювання складних систем.

В цілому, ця лабораторна робота надає студентам можливість поєднати теоретичні знання з практичними навичками використання Matlab для аналізу та розв'язання складних системних задач. Вона сприяє розвитку аналітичних та програмних навичок у контексті моделювання динамічних систем та штучного інтелекту.